Documento de Diseño (D2)

CascoVision · Sistema de detección automatizada de EPP mediante visión computacional

Iahn Vera · Joaquín Armijo · Benjamín Cáceres · Fernando Muñoz

Capstone PTY4614 · 2026.02 · Versión 0.9.1 (lista para aprobación) · 23-10-2026

Índice

Control del documento 2
1. Introducción 2
1.1 Propósito 2
1.2 Resumen de la solución 3
2. Arquitectura 3
2.1 Decisiones de arquitectura 4
2.2 Componentes 4
2.3 Comunicación entre servicios 4
2.4 Presupuesto de latencia de la alerta (RNF-02) 5
3. Despliegue 6
3.1 Nodos 6
3.2 Zonas de confianza 6
3.3 Diseño Docker 7
4. Modelo de datos 8
4.1 Criterios 8
4.2 Tablas 8
5. Diagramas UML 10
5.1 Clases 10
5.2 Secuencia de la funcionalidad principal 11
5.3 Componentes 12
6. Protocolo de la baliza 12
6.1 POST /alert 12
6.2 Política del agente y *heartbeat* 13
7. Mockups del dashboard 13
8. Trazabilidad de requisitos a componentes 15
9. Brechas con el código actual y pendientes 16

9.1 Cambios del código en qa que este diseño incorpora 16
10. Aprobación 17

Control del documento

Versión	Fecha	Autor	Descripción
0.7	15-10-2026	Equipo CascoVision	Arquitectura resumida en el README y en la presentación de la semana 10
0.8	20-10-2026	Iahn Vera	Capítulo de arquitectura y despliegue (tarea 3.1)
0.9	21-10-2026	Iahn Vera	Consolidación (tarea 3.8): protocolo de la baliza (3.3), diagramas UML (3.5), y los borradores del modelo de datos (3.2), los mockups (3.4) y el diseño Docker (3.6)
0.9.1	23-10-2026	Iahn Vera	Las partes 3.2, 3.4 y 3.6 ya están en qa con la versión de sus autores; migraciones renumeradas (§4.1) y cambios del código de qa (§9.1)
1.0	*pendiente*	Equipo CascoVision	Versión aprobada (puerta de fase 3 · Diseño)

Metodología: tradicional en cascada. Este documento cierra la **fase 3 (Diseño)**. Su entrada es el SRS v0.4 (D1) y su salida alimenta la construcción (fase 4) y el plan de pruebas (D3).

Estado de los capítulos:

Capítulo	Tarea	Responsable	Estado
§2 Arquitectura y §3 despliegue	3.1	Iahn Vera	Terminado
§3.3 Diseño Docker	3.6	Joaquín Armijo	Terminado (en qa, PR #8)
§4 Modelo de datos	3.2	Fernando Muñoz	Terminado (en qa, PR #9)
§5 Diagramas UML	3.5	Iahn Vera	Terminado
§6 Protocolo de la baliza	3.3	Iahn Vera	Terminado
§7 Mockups	3.4	Joaquín Armijo	Terminado (en qa, PR #8)

El detalle completo de la arquitectura está en el capítulo *D2 · Arquitectura y despliegue* (v1.0) y el del protocolo en docs/PROTOCOLO_BALIZA.md. Este documento presenta la versión consolidada.

1. Introducción

1.1 Propósito

El D2 describe **cómo se construye CascoVision**: su arquitectura, cómo se despliega, qué datos guarda, cómo se comportan sus clases y componentes,

cómo se comunica con la baliza y cómo se ve su interfaz. Todo requisito del SRS se asigna aquí a un componente (§8), y cada componente tiene casos de prueba en el D3.

1.2 Resumen de la solución

Las **cámaras IP existentes** entregan video a un **agente Edge** que corre en un servidor dentro de la faena. El agente analiza 1 cuadro por segundo con **YOLO26s** y **ByteTrack**. Si detecta a una persona sin casco o sin chaleco dentro de una zona de riesgo, **activa directamente la baliza zonal** (M5Stack StickS3), que suena y muestra la infracción. Después difumina los rostros, guarda la evidencia e informa a la **API REST**, que registra el incidente en **PostgreSQL**, lo muestra en el **dashboard** y notifica al prevencionista por **SMS y correo**. Todo se despliega con **Docker Compose** y el video nunca sale de la obra.

2. Arquitectura

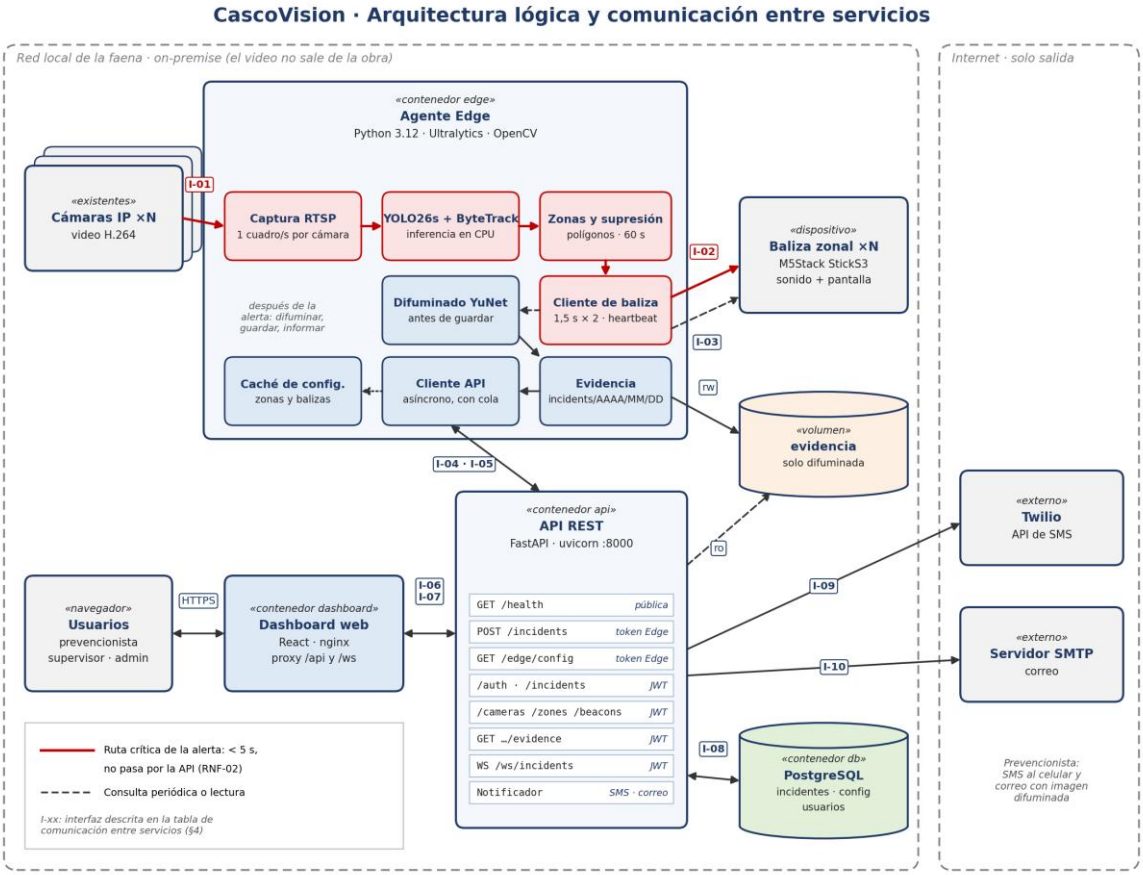


Figura 1. Arquitectura lógica de CascoVision. La línea roja es la ruta crítica de la alerta.

2.1 Decisiones de arquitectura

ID	Decisión	Motivo	Requisitos
AD-01	Todo corre en un servidor Edge dentro de la faena; el video no sale de la obra	Privacidad (Ley N° 19.628) y latencia	RNF-08, RNF-09
AD-02	El agente Edge activa la baliza directamente, sin pasar por la API	Ahorra un salto de red y alerta aunque la API esté caída	RF-07, RNF-02
AD-03	La alerta va antes que el difuminado, la evidencia y el registro	Solo lo necesario queda en la ruta crítica	RNF-02
AD-04	El agente informa a la API de forma asíncrona, con cola en disco	Una API lenta no detiene el análisis ni las alertas	RNF-13
AD-05	HTTP/REST con JSON entre componentes y WebSocket hacia el dashboard	Simple, conocido por el equipo y nativo en FastAPI	RF-13, RNF-04
AD-06	Evidencia en un volumen del servidor; la base de datos guarda la ruta y la API la entrega con JWT	Sin imágenes en la base de datos ni rutas expuestas	RF-05, RF-19
AD-07	Un único puerto publicado (443, nginx del dashboard)	Menor superficie expuesta	RNF-07, RNF-10
AD-08	Inferencia en CPU; entrenamiento con GPU en Colab	El cliente no compra GPU	RNF-01
AD-09	Docker Compose en un equipo; más agentes Edge si crecen las cámaras	Instalación con un comando y crecimiento por configuración	RNF-12, RNF-15
AD-10	Almacenamiento intercambiable en la API (PostgreSQL o JSON para pruebas)	Ya implementado en `almacen.py`	RNF-17

2.2 Componentes

Componente	Responsabilidad	Tecnología
Agente Edge	Captura, detección, zonas, supresión, alerta a la baliza, difuminado, evidencia y reporte a la API	Python 3.12, Ultralytics (YOLO26s, ByteTrack), OpenCV (YuNet)
Baliza zonal	Suena y muestra la zona y el EPP faltante; responde al *heartbeat*	M5Stack StickS3, firmware con M5Unified
API REST	Incidentes, configuración, usuarios, evidencia, tiempo real y notificaciones	FastAPI, uvicorn, psycopg 3
Base de datos	Incidentes, cámaras, zonas, balizas, usuarios y roles	PostgreSQL 16
Dashboard	Monitoreo, gestión de incidentes, reportes y configuración	React servido con nginx
Servicios externos	SMS y correo al prevencionista	Twilio y SMTP

2.3 Comunicación entre servicios

ID	Origen → destino	Protocolo	Autenticación	Espera

ID	Origen → destino	Protocolo	Autenticación	Espera
I-01	Cámara → agente	RTSP, :554	Credenciales de la cámara (variables de entorno)	Continua, con reconexión
I-02	Agente → baliza	HTTP POST /alert, :80	Token de la baliza	1,5 s y un reintento
I-03	Agente → baliza	HTTP GET /health, :80	Pública	Cada 30 s
I-04	Agente → API	HTTP POST /incidents, :8000	Token del agente Edge	Asíncrono, con cola
I-05	Agente ↔ API	HTTP GET /edge/config, PUT /edge/beacons/{id}/status	Token del agente Edge	Cada 60 s
I-06	Dashboard → API	HTTPS, :443 → :8000	JWT (≤ 8 h)	Por acción
I-07	API → dashboard	WebSocket seguro	JWT	≤ 3 s
I-08	API → PostgreSQL	TCP, :5432	Usuario y clave	Por petición
I-09	API → Twilio	HTTPS, :443	Credenciales de Twilio	≤ 30 s
I-10	API → SMTP	SMTP con STARTTLS, :587	Usuario y clave	≤ 30 s

2.4 Presupuesto de latencia de la alerta (RNF-02)

Etapa	Presupuesto
Espera del cuadro (1 FPS)	≤ 1,0 s
Inferencia y seguimiento en CPU	≤ 0,5 s (se mide en la tarea 4.3)
Zonas y supresión	< 0,01 s
Orden a la baliza (1,5 s × 2 intentos)	≤ 3,0 s
Total en el peor caso	**≤ 4,5 s, bajo el límite de 5 s**

3. Despliegue

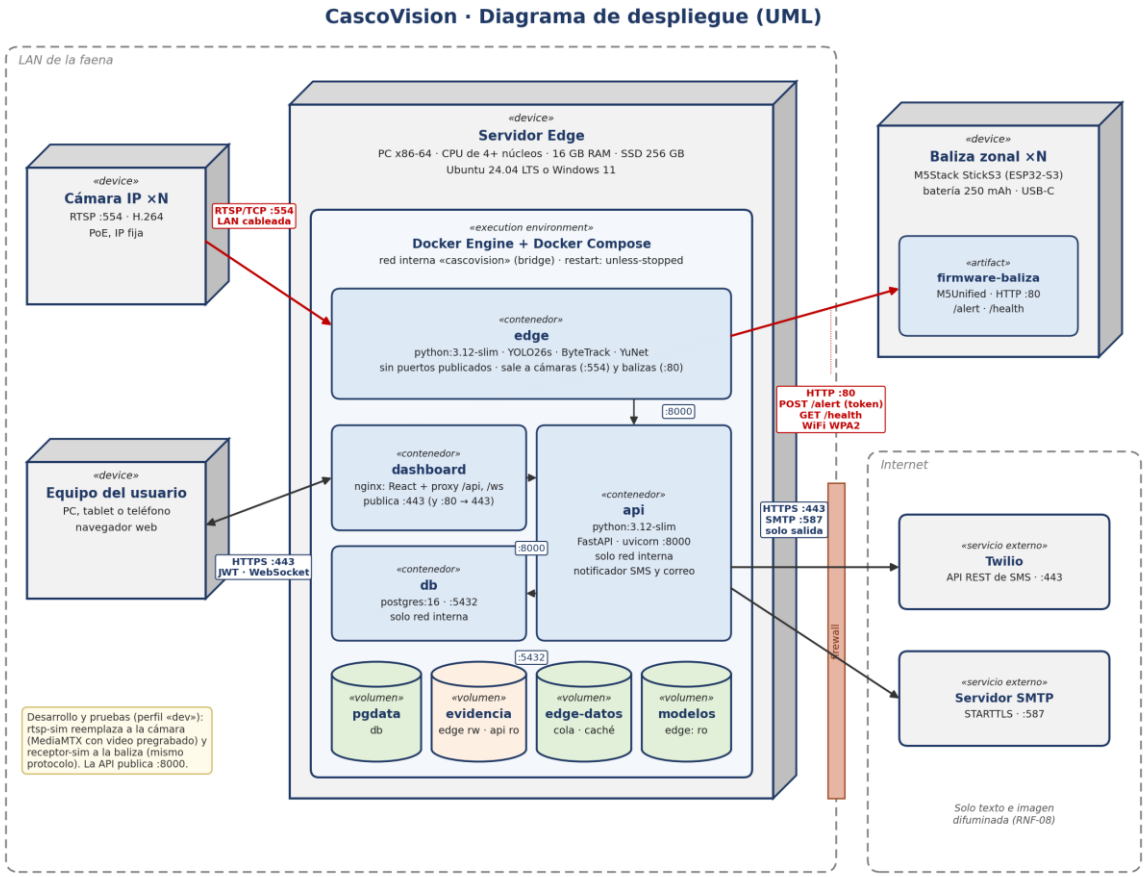


Figura 2. Diagrama de despliegue UML de CascoVision en la red de la faena.

3.1 Nodos

Nodo	Características	Qué ejecuta
Servidor Edge	Mínimo propuesto: CPU de 4 núcleos, 16 GB de RAM, SSD de 256 GB; Ubuntu 24.04 o Windows 11 con Docker	Los 4 contenedores y sus volúmenes
Cámara IP	RTSP, H.264, PoE, IP fija	Su firmware
Baliza zonal (una por zona)	StickS3, WiFi de 2,4 GHz, batería de 250 mAh	Firmware de la baliza, HTTP en el puerto 80
Equipo del usuario	Navegador web	Dashboard
Twilio y SMTP	En Internet; solo reciben conexiones salientes	SMS y correo

3.2 Zonas de confianza

Desde Internet no se abre ningún puerto hacia la faena; solo la API sale a Twilio y SMTP. En la LAN solo se publica el puerto 443 del servidor. Las balizas y las

cámaras van en una red WiFi de dispositivos separada (WPA2), porque la orden a la baliza viaja por HTTP sin cifrar.

3.3 Diseño Docker

Servicio	Imagen	Puertos	Volúmenes
db	postgres:16-alpine	5432, solo red interna	pgdata
api	python:3.12-slim	8000, solo red interna	evidencia (solo lectura)
dashboard	nginx con React compilado en una etapa previa con Node	443 y 80 publicados	certificados
edge	python:3.12-slim con Ultralytics de CPU y OpenCV sin interfaz	Ninguno	evidencia, edge-datos, modelos (solo lectura)
rtsp-sim (perfil dev)	MediaMTX con un video pregrabado	8554, interno	
receptor-sim (perfil dev)	python:3.12-slim con el protocolo de la baliza	80, interno	

- **Red:** una red bridge interna (cascovision). Los servicios se llaman por nombre (api:8000, db:5432).
- **Arranque:** db sano → api (aplica migraciones) → dashboard. El agente edge no espera a la API: arranca con la configuración guardada.
- **Salud:** pg_isready, GET /health, respuesta de nginx y un archivo de latido del agente. Todos con restart: unless-stopped (RNF-13).
- **Secretos:** solo en .env, que no se sube al repositorio (RNF-10). Al .env.example actual le faltan API_URL, CASCOVISION_EVIDENCIA, EDGE_COLA, RTSP_USER, RTSP_PASSWORD y TZ.
- **Comandos:** docker compose up -d levanta el sistema; docker compose --profile dev up -d agrega los simuladores.

4. Modelo de datos

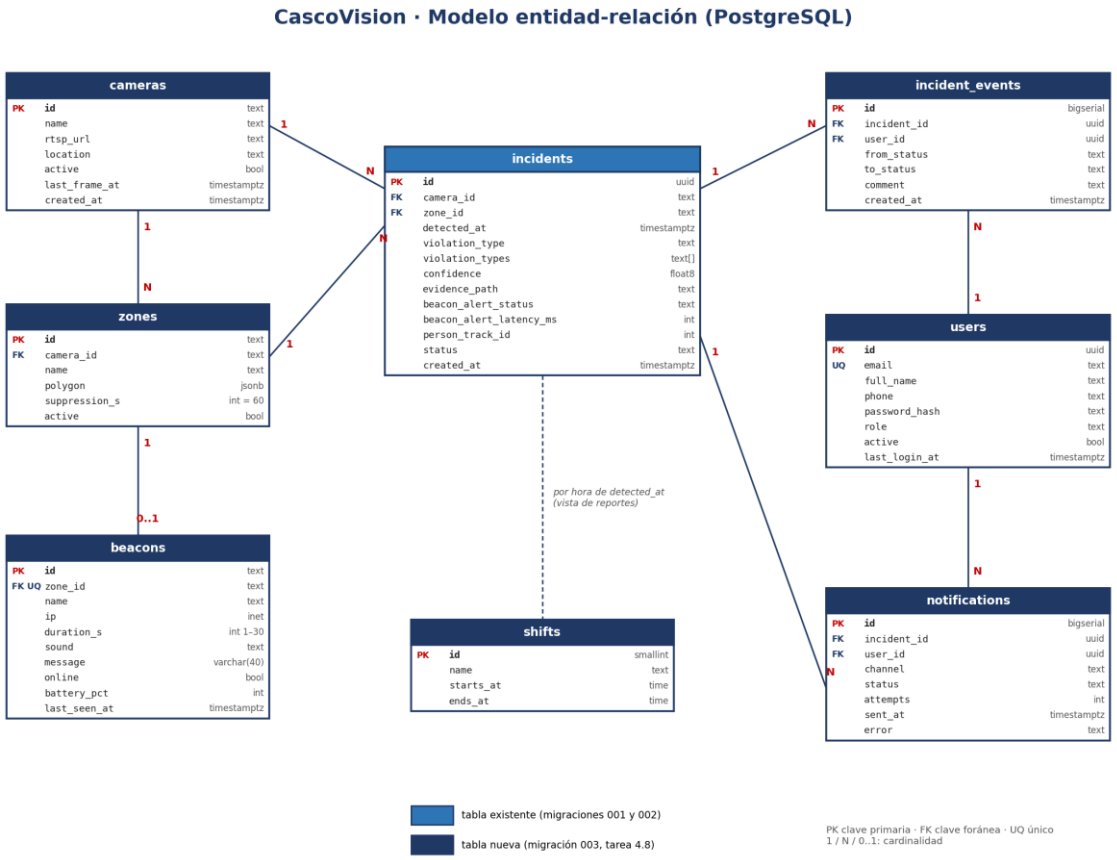


Figura 3. Modelo entidad-relación (PostgreSQL).

4.1 Criterios

- Se conserva la tabla incidents de las migraciones 001 y 002, que la API ya usa. La migración 003 ya existe en qa y agrega a incidents el ciclo de vida del episodio (§9.1). Por eso las tablas nuevas entran en la migración **004** (tarea 4.8) y las claves foráneas de incidents en la **005**; docs/MODELO_DATOS.md debe ajustar esa numeración.
- Cámaras, zonas y balizas usan identificadores de texto legibles (zona-norte), los mismos que ya envía el agente.
- No se guardan datos personales de los trabajadores: el número de seguimiento es anónimo y la evidencia está difuminada.
- Las credenciales de las cámaras y los tokens no van en la base de datos.

4.2 Tablas

Tabla	Contenido	Columnas principales	Requisitos
cameras	Cámaras de la faena	id, name, rtsp_url (sin credenciales), active,	RF-01, RF-18

Tabla	Contenido	Columnas principales	Requisitos
		last_frame_at	
zones	Zonas de riesgo	id, camera_id, name, polygon (jsonb, coordenadas de 0 a 1), suppression_s	RF-04, RF-08
beacons	Balizas	id, zone_id (único), ip, duration_s (1–30), sound, message, online, battery_pct	RF-09, RF-10, RNF-14
incidents	Incidentes (existente)	id, camera_id, zone_id, detected_at, violation_type(s), confidence, evidence_path, beacon_alert_status, person_track_id, status	RF-05, RF-14
incident_events	Historial de estados	incident_id, user_id, from_status, to_status, comment	RF-14
users	Usuarios	id, email (único), phone, password_hash (bcrypt), role	RF-16, RF-17, RNF-06
notifications	Envíos de SMS y correo	incident_id, user_id, channel, status, attempts, sent_at	RF-11, RF-12, RNF-03
shifts	Turnos para reportes	name, starts_at, ends_at	RF-15

El diccionario de datos completo, con tipos, nulos, restricciones e índices, está en el borrador de la tarea 3.2.

5. Diagramas UML

5.1 Clases

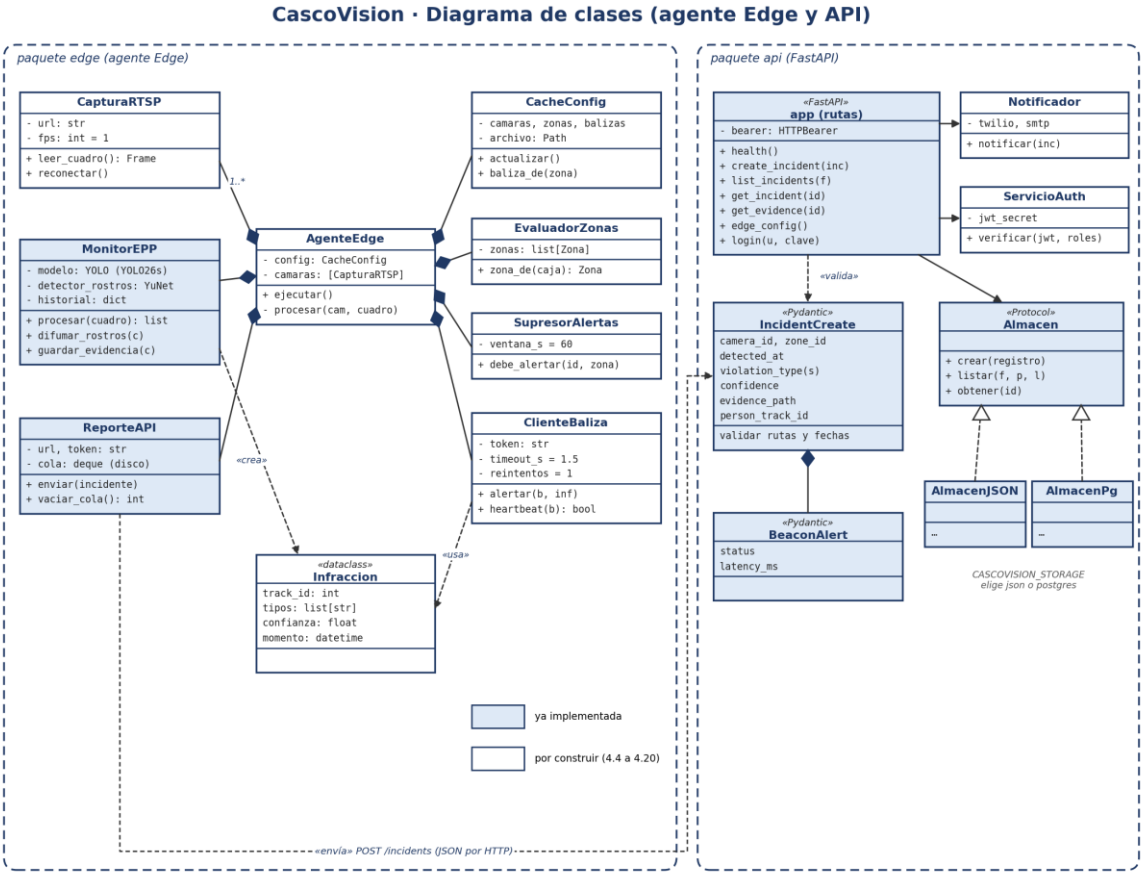


Figura 4. Diagrama de clases del agente Edge y de la API. En azul, las clases ya implementadas.

El agente Edge se organiza alrededor de AgenteEdge, que compone una clase por responsabilidad, para poder probarlas por separado. MonitorEPP y ReporteAPI ya existen en el código, en Yolo/modelamiento/. En la API, las rutas validan la entrada con modelos Pydantic y guardan a través de la interfaz Almacen, con dos implementaciones (JSON y PostgreSQL) que ya existen.

5.2 Secuencia de la funcionalidad principal

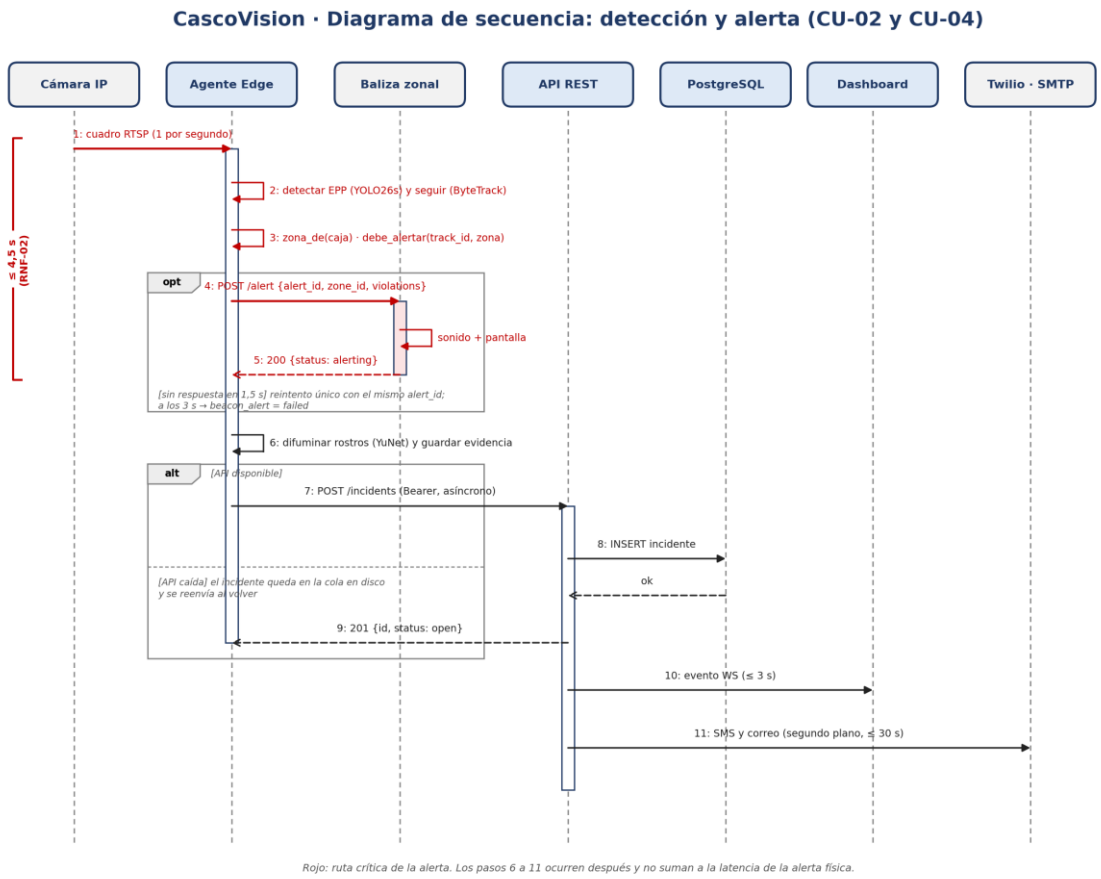


Figura 5. Secuencia de detección y alerta (CU-02 y CU-04).

Los pasos 1 a 5 forman la ruta crítica (≤ 4,5 s). Si la baliza no responde en 1,5 s, el agente reintenta una vez; a los 3 s la alerta queda «fallida». Si la API está caída, el incidente queda en la cola en disco y se reenvía al volver.

5.3 Componentes

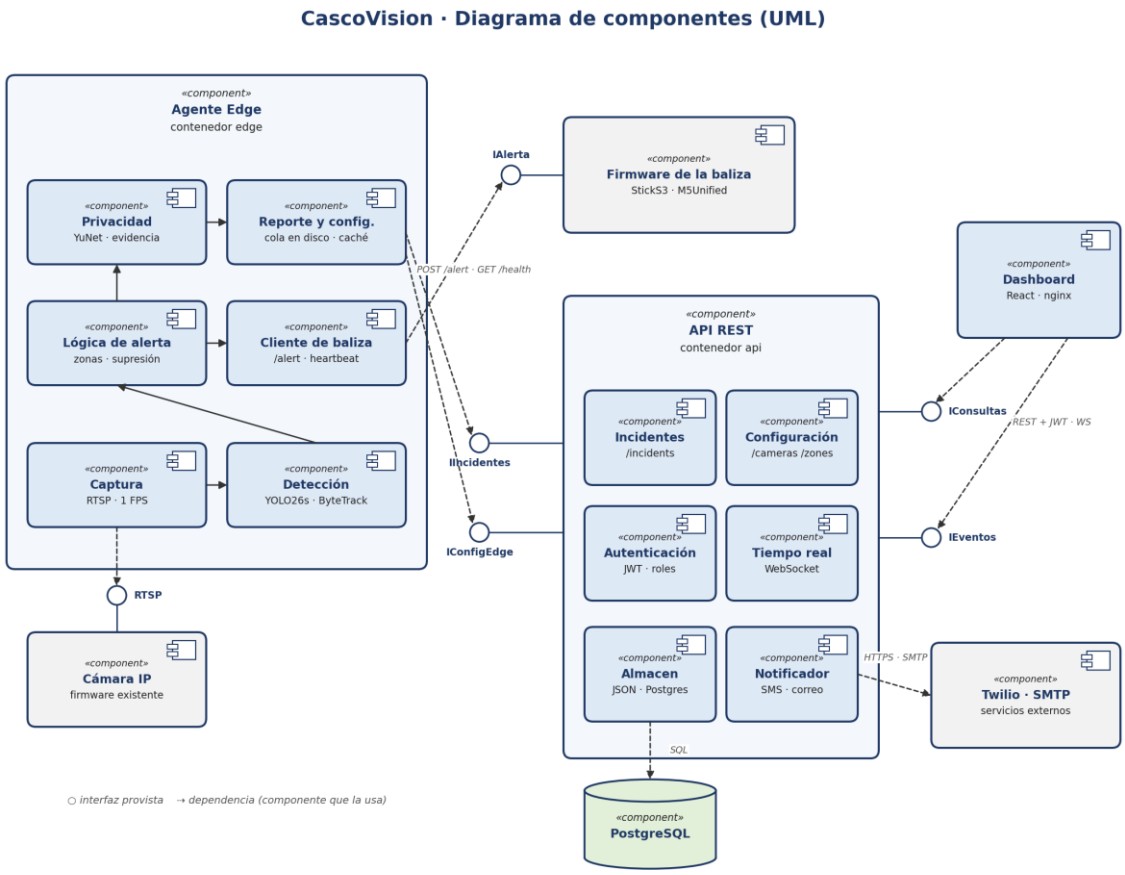


Figura 6. Diagrama de componentes y sus interfaces.

6. Protocolo de la baliza

Contrato entre el agente Edge y la baliza, que cumplen por igual el firmware del StickS3 (tarea 4.7) y el receptor simulado (tarea 4.6). El documento completo es docs/PROTOCOLO_BALIZA.md.

6.1 POST /alert

Campo	Validación
alert_id	UUID; un reintento usa el mismo valor y no duplica la alerta
zone_id, zone_name	Zona de la baliza (si no coincide responde 409); nombre de hasta 20 caracteres
violations	missing_helmet y/o missing_reflective_vest
duration_s	Entre 1 y 30
sound, message	siren, voice o beep; texto de hasta 40 caracteres
sent_at	ISO 8601 con zona horaria

Exige Authorization: Bearer <BALIZA_TOKEN>. Respuestas: 200 (alerta en curso), 400 (campo inválido), 401 (token), 409 (otra zona) y 413 (cuerpo mayor que 1 KB). La baliza responde en menos de 200 ms y suena en segundo plano.

6.2 Política del agente y *heartbeat*

Situación	Acción	Resultado registrado
200	Mide el tiempo de ida y vuelta	alerted
Sin respuesta en 1,5 s	Un reintento con el mismo alert_id	alerted o, a los 3 s, failed
400, 401, 409 o 413	No reintenta	failed
Baliza fuera de línea	No envía la orden	skipped_offline

GET /health es público y devuelve el estado, la batería, la señal y si está alertando. El agente lo consulta cada 30 s; tras 3 fallos seguidos la baliza queda fuera de línea, y el agente informa los cambios a la API.

7. Mockups del dashboard

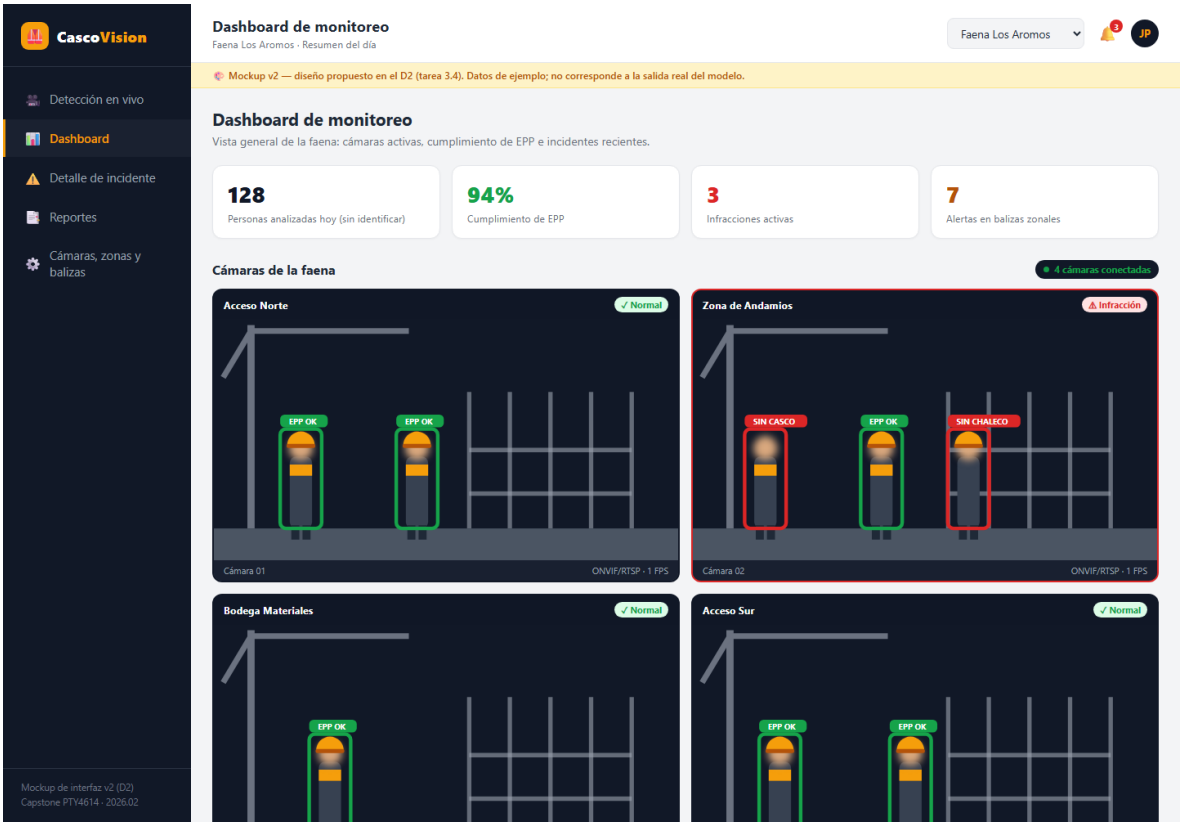


Figura 7. Dashboard de monitoreo.

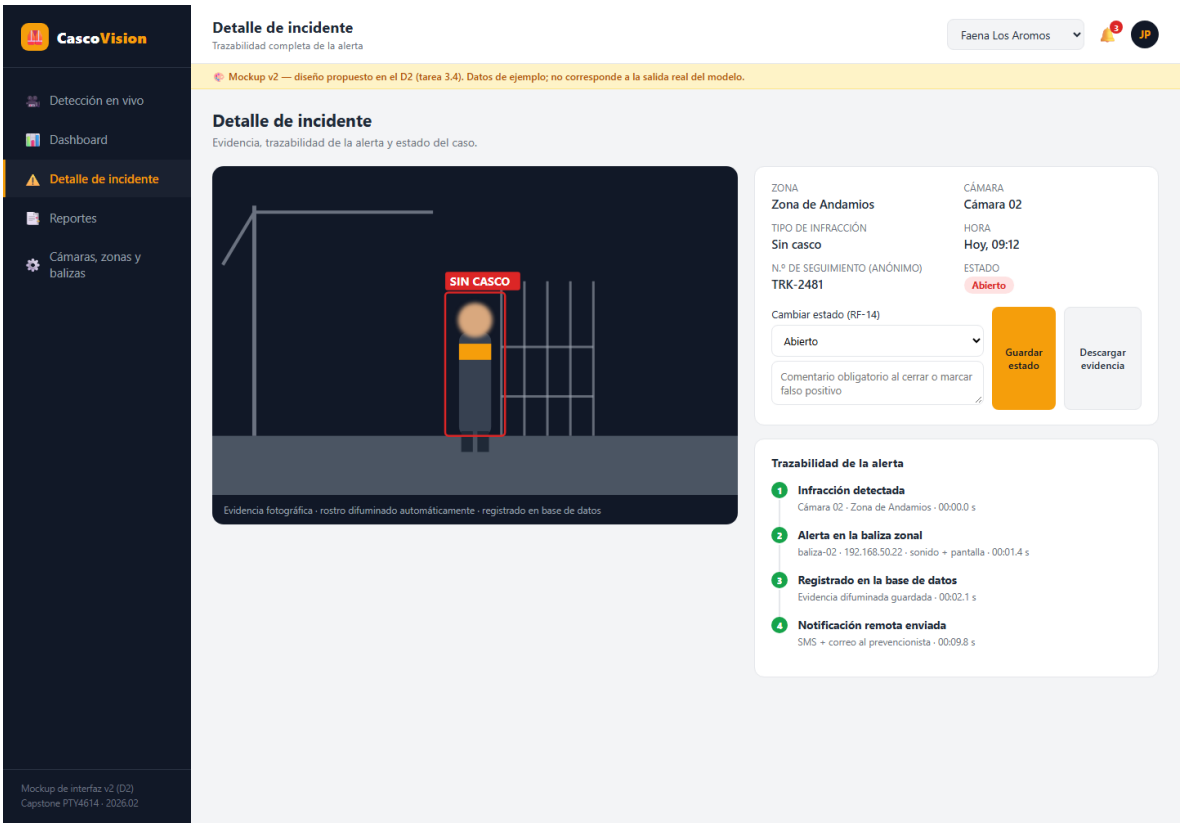


Figura 8. Detalle de incidente con evidencia difuminada, cambio de estado y trazabilidad de la alerta.

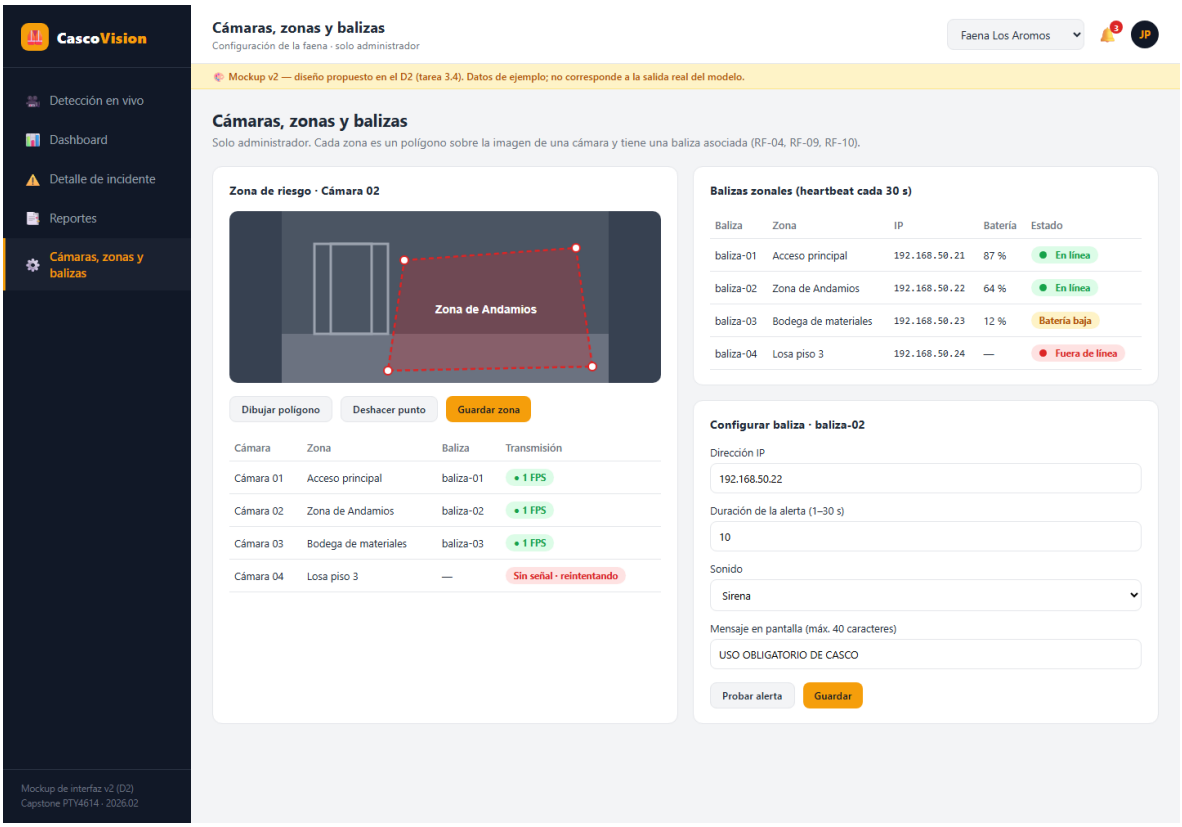


Figura 9. Configuración de cámaras, zonas y balizas (solo administrador).

Vista	Usuario	Requisitos
Inicio de sesión	Todos	RF-17
Detección en vivo	Prevencionista, supervisor	RF-02, RF-03, RF-06
Dashboard de monitoreo	Prevencionista, supervisor	RF-13, RNF-04
Detalle de incidente	Prevencionista	RF-05, RF-14, RF-19
Reportes	Prevencionista, supervisor	RF-15
Cámaras, zonas y balizas	Administrador	RF-04, RF-09, RF-10, RF-18

El mockup interactivo (CascoVision_Mockup_v2.html) actualiza el de la Fase 1. La baliza reemplaza al dispositivo vibratorio, se agregan el inicio de sesión, la configuración y la gestión de estados, y no se identifica a las personas.

8. Trazabilidad de requisitos a componentes

Requisitos	Componente	Diseño
RF-01, RF-18	Agente Edge · captura	I-01, cameras
RF-02, RF-03, RNF-05	Agente Edge · detección	MonitorEPP (YOLO26s)
RF-04, RF-08	Agente Edge · zonas y supresión	EvaluadorZonas, SupresorAlertas, zones
RF-05, RF-06, RF-19	Agente Edge · privacidad y	AD-06, volumen evidencia,

Requisitos	Componente	Diseño
	evidencia; API	incidents
RF-07, RNF-02, RNF-11, RNF-14	Agente Edge · cliente de baliza; firmware	§6, I-02, I-03
RF-09, RF-10	API · configuración; dashboard	beacons, vista de configuración
RF-11, RF-12, RNF-03	API · notificador	I-09, I-10, notifications
RF-13, RNF-04	API · tiempo real; dashboard	I-07 (WebSocket)
RF-14	API · incidentes; dashboard	incident_events, vista de detalle
RF-15	API · reportes	shifts, vistas de reportería
RF-16, RF-17, RNF-06, RNF-07	API · autenticación	users, ServicioAuth, JWT
RNF-08, RNF-09, RNF-10	Toda la solución	AD-01, AD-07, §3.2
RNF-12, RNF-13, RNF-15	Despliegue	AD-04, AD-09, §3.3

9. Brechas con el código actual y pendientes

ID	Brecha	Tarea
B-01	El agente provisional envía a la API dentro del ciclo de análisis y se bloquea hasta 40 s si la API cae	4.4
B-02	La captura no usa RTSP a 1 FPS	4.4
B-03	La cámara y la zona están fijas en el código, sin polígonos	4.4, 4.15
B-04	No existe el cliente de baliza	4.4, 4.6, 4.7
B-05	Faltan en la API el JWT, la configuración, la evidencia, el WebSocket y el notificador	4.15, 4.16, 4.19, 4.20
B-06	No hay Docker y el agente no está en src/edge/	4.17
B-07	Rutas de evidencia y cola fijas en el código	4.4
B-08	GET /config solo informa si la API leyó el .env; falta GET /edge/config con cámaras, zonas y balizas (I-05)	4.15

9.1 Cambios del código en qa que este diseño incorpora

Al 23-10-2026, qa (e025a4b) trae cambios de la construcción que no contradicen el diseño, pero lo precisan:

ID	Cambio	Efecto en el diseño	Origen
C-01	El detector pasa a **5 clases** (helmet, no_helmet, no_vest, Person, vest): la falta de EPP se detecta en forma positiva	MonitorEPP ya no infiere la infracción por ausencia	Tarea 4.2, train-10
C-02	Cada infracción es un **episodio** con episode_id y fases inicio, ping (cada 60 s) y fin	SupresorAlertas reenvía el mismo incidente en vez de silenciarlo; la API lo actualiza sin duplicarlo	Tarea 4.5, migración 003
C-03	Si se pierde de vista a la	Evita duplicar el incidente cuando	Tarea 4.5

ID	Cambio	Efecto en el diseño	Origen
	persona, el episodio queda en espera 15 s antes de cerrarse	el seguimiento cambia de identificador	
C-04	Si no se detecta a la persona pero sí EPP, se difumina el cuadro completo	La privacidad (RF-06) no depende de que se detecte a la persona	Tarea 4.5

A confirmar en la reunión de aprobación:

1. El agente Edge hace el *heartbeat* de las balizas.
2. WebSocket para el dashboard.
3. Requisito mínimo del servidor Edge.
4. Red separada para balizas y cámaras.
5. Retención de la evidencia (propuesta: 90 días).
6. Una baliza por zona.

10. Aprobación

Integrante	Rol	Aprobación	Fecha
Iahn Vera	Jefe de proyecto · IA y agente Edge	Pendiente	
Joaquín Armijo	Dev fullstack	Pendiente	
Fernando Muñoz	Dev BBDD y baliza	Pendiente	
Benjamín Cáceres	QA	Pendiente	